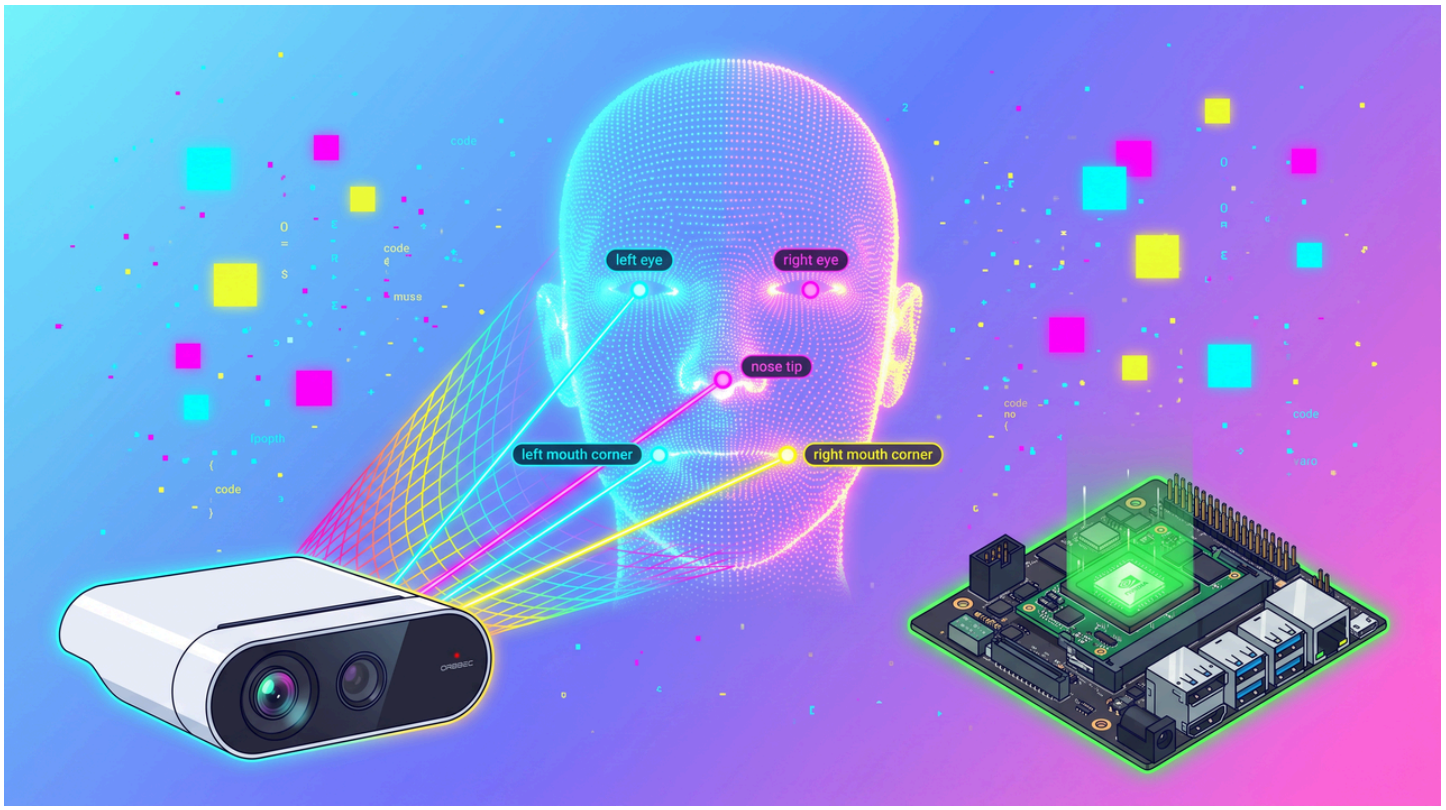
# Jetson Orin Nano + Orbbec Femto Mega 复现 RGB-D 人脸活体检测项目手把手指南 🚀




> 本文记录的是一次真实复现过程：从原项目几乎没有 README，到在 Jetson Orin Nano 上跑通 RGB-D 相机、人体分割、人脸活体判断。中间踩到的坑不少，但好消息是：每个坑都有办法绕过去。走，我们一点点把它跑起来。😄
>
> 🔗GitHub

## 1. 项目目标

我们要复现的项目叫：

```
Facial-authenticity-judgment
```

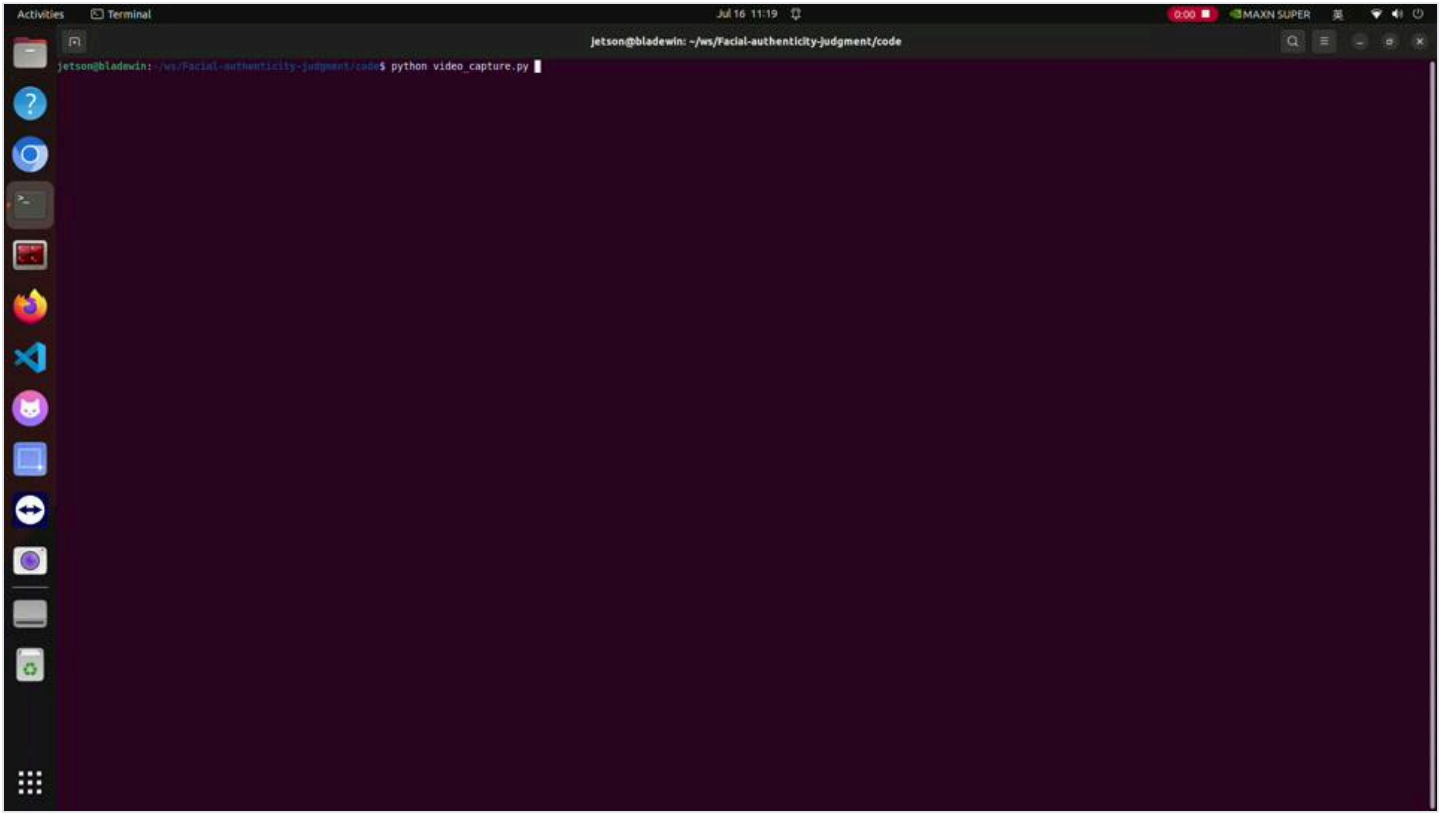
它的核心功能是：

- 使用 Orbbec 深度相机获取 RGB 图和深度图
- 使用 YuNet 检测人脸和 5 个关键点
- 读取 5 个关键点对应的深度值
- 用深度标准差判断是真人脸还是平面攻击

简单说就是：

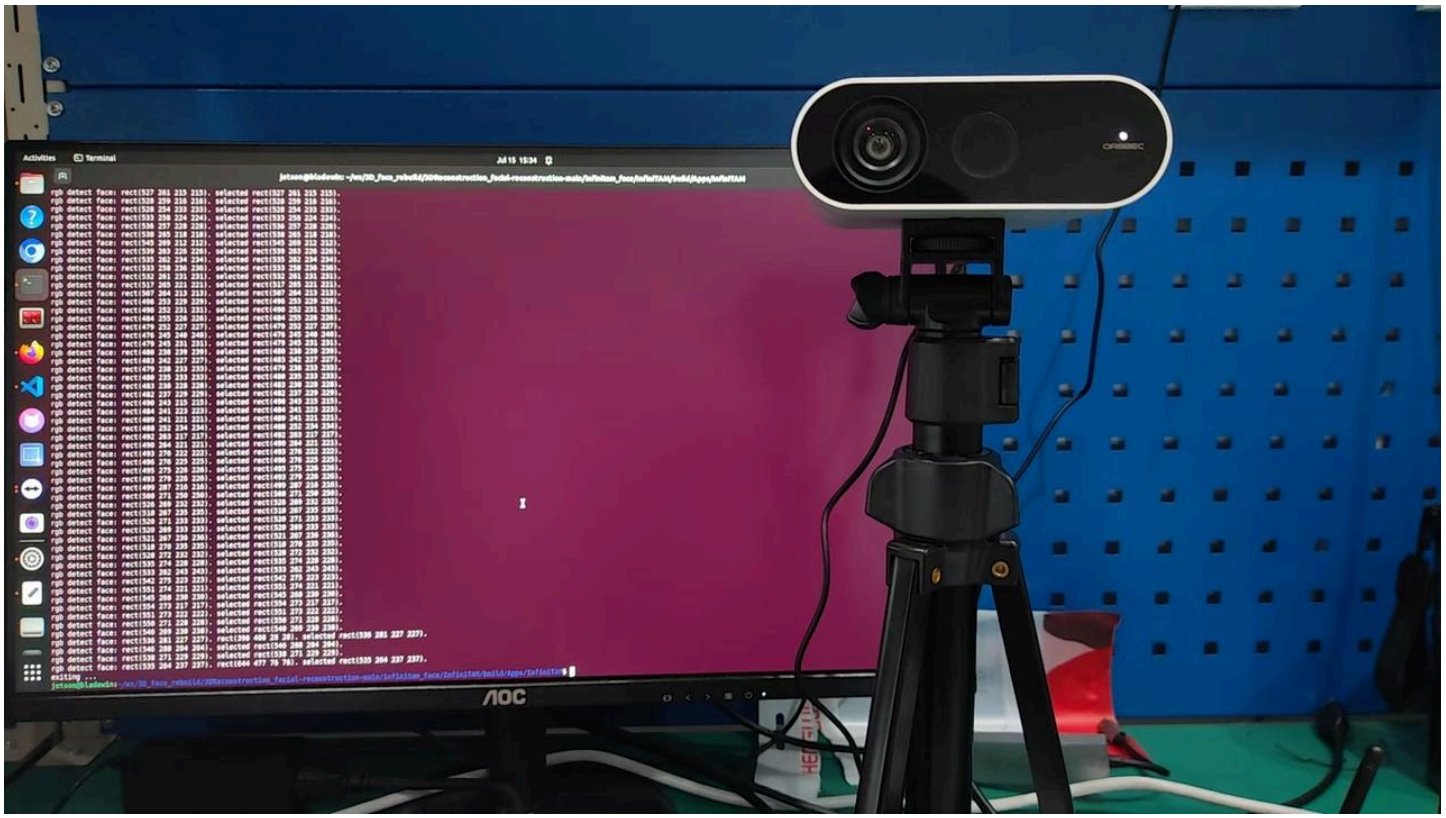
**如果脸是立体的，鼻子、眼睛、嘴角等点到相机的距离应该有变化；如果是照片或屏幕，几个点的深度会比较接近。**

当然，这只是教学级/实验级活体检测逻辑，不是生产级安全算法。它更适合理解 RGB-D 活体检测流程。




## 2. 硬件和系统环境

本次复现环境：




| 项目 | 配置 |
| --- | --- |
| 开发板 | NVIDIA Jetson Orin Nano |
| 系统 | Ubuntu 22.04 / JetPack 6.x |
| 相机 | Orbbec Femto Mega |
| Python | Python 3.x |
| 图像库 | OpenCV 4.x |
| 相机 SDK | pyorbbecsdk2 |

## 3. 原项目的问题

原项目代码很小，主要包含：

代码块

```
code/
├── video_capture.py
├── body_segmentation.py
```

```
├── anti-spoofing.py
├── face_detection_yunet_2022mar.onnx
└── 示例图片
```

原始代码用的是：

代码块

```
cv.VideoCapture(0, cv.CAP_OBSENSOR)
```

这在普通 OpenCV 环境下看起来很优雅，但在 Jetson + Femto Mega 上问题比较多：

- `CAP_OBSENSOR` 后端可能没有正确编译
- `grab()` 会一直失败
- 硬件 depth-to-color 对齐可能启动成功但不出帧
- OpenCV 对 Orbbec 不同设备/固件组合的兼容性不够稳定

所以我们最终决定：

****不用 OpenCV 的 OBSENSOR 后端，改用 Orbbec 官方 Python SDK：`pyorbbecsdk2`。****

## 4. 安装依赖

进入项目代码目录：

代码块

```
cd ~/ws/Facial-authenticity-judgment/code
```

安装 Python 依赖：

代码块

```
pip install -r requirements.txt
```

`requirements.txt` 内容如下：

代码块

```
numpy
opencv-python
pyorbbecsdk2
```

如果你还没有新版 YuNet 模型，运行：

代码块

```
python download_yunet_model.py
```

如果 Jetson 网络环境不方便，也可以手动下载：

代码块

```
wget -O face_detection_yunet_2023mar.onnx \
https://github.com/opencv/opencv_zoo/raw/main/models/face_detection_yunet/face_detection_yunet_2023mar.onnx
```

## 5. 代码迁移思路

我们新增了一个公共采集模块：

```
代码块
1   orbbec_capture.py
```

它负责：

- 初始化 Orbbec Pipeline
- 获取 Color Frame
- 获取 Depth Frame
- 将 RGB 转成 OpenCV BGR
- 将 Depth 转成毫米单位深度图
- 将 RGB 和 Depth 统一调整成 `640x480`
- 处理相机模式 fallback

这样三个业务脚本就不用各自打开相机了：

```
代码块
1   video_capture.py        # 只负责预览
2   body_segmentation.py    # 只负责分割
3   anti-spoofing.py        # 只负责人脸检测和活体判断
```

这一步非常关键。

**相机采集是一切后续功能的地基，地基稳了，后面才好调。**

## 6. 第一个坑：OpenCV 的 CAP_OBSENSOR 抓不到帧 

最开始运行原项目：

```
代码块
```

```
python video_capture.py
```

现象是：

代码块

```
Fail to grab data from camera.
Fail to grab data from camera.
Fail to grab data from camera.
```

说明相机对象可能打开了，但拿不到实际 RGB-D 帧。

解决方案：

代码块

```
from pyorbbecsdk import Pipeline
```

改用 `pyorbbecsdk2` 直接获取 Orbbec 数据流。

## 7. 第二个坑：硬件 D2C 对齐会卡住 😵

迁移到 `pyorbbecsdk2` 后，第一次尝试硬件 depth-to-color 对齐：

代码块

```
Using hardware depth-to-color alignment.
Fail to grab data from camera.
```

它看起来启动成功了，但一直没有有效帧。

最后采用默认策略：

```
代码块
1   software depth-to-color alignment
```

也就是软件对齐。

运行时你会看到：

```
代码块
1   Using software depth-to-color alignment.
2   Output RGB-D frame size: 640x480
```

如果你想切换模式，可以用环境变量：

```
代码块
1   ORBBEC_ALIGN_MODE=software python video_capture.py
2   ORBBEC_ALIGN_MODE=none python video_capture.py
3   ORBBEC_ALIGN_MODE=default python video_capture.py
4   ORBBEC_ALIGN_MODE=auto python video_capture.py
```

建议优先用：

```
代码块
```

```
ORBBEC_ALIGN_MODE=software
```

## 8. 第三个坑：数据流太大，Jetson 上处理卡顿 🐢

能出图之后，运行人体分割脚本时又遇到卡顿。

我们做了两件事：

### 8.1 统一输出 640x480

在 `orbbec_capture.py` 中统一 resize：

- RGB resize 到 `640x480`
- Depth resize 到 `640x480`
- Depth 使用最近邻插值，避免毫米值被平滑污染

默认输出：

代码块

```
640x480
```

也可以临时改成更小尺寸：

代码块

```
ORBBEC_OUTPUT_SIZE=320x240 python body_segmentation.py
```

### 8.2 去掉每帧保存图片

原人体分割脚本每帧都会：

```
代码块
1   cv.imwrite("depth.jpg", ...)
2   cv.imwrite("segment.jpg", ...)
3   cv.imwrite("contour.jpg", ...)
```

这在 Jetson 上很容易拖死实时显示。

现在默认只显示，不每帧保存。

如果确实要保存，可以在代码里调整：

```
代码块
1   SAVE_EVERY_N_FRAMES = 30
```

意思是每 30 帧保存一次。

## 9. 第四个坑：两个脚本抢相机设备 🔒

如果先运行：

```
代码块
1   python video_capture.py
```

然后不退出，又运行：

```
代码块
1   python body_segmentation.py
```

会报：

```
代码块
1   Device or resource busy
```

用命令查看：

```
代码块
1   sudo fuser -v /dev/video*
```

可以看到类似：

```
代码块
1   /dev/video0:  jetson  8648  python
2   /dev/video1:  jetson  8648  python
```

再查进程：

```
代码块
```

```
ps -ef | grep -E "video_capture|body_segmentation|anti-spoofing"
```

发现：

代码块

```
python video_capture.py
```

这说明相机已经被 `video_capture.py` 占住了。

解决方案：

代码块

```
pkill -f video_capture.py
```

或者回到运行脚本的终端，按：

代码块

```
q
```

或：

代码块

```
Esc
```

结论很重要：

**同一时间只能有一个脚本打开相机。**

如果未来想同时显示 RGB、分割、活体结果，就应该写一个统一主程序，只打开一次相机，然后在同一个循环里做多种处理。

## 10. 第五个坑：旧 YuNet 模型和 OpenCV 4.10 不兼容 🤯

运行活体检测：

```
代码块
1   python anti-spoofing.py
```

遇到错误：

```
代码块
1   Layer with requested id=-1 not found
```

原因是原项目的模型：

```
代码块
1   face_detection_yunet_2022mar.onnx
```

在 OpenCV 4.10 上可能不兼容。

解决方法是下载新版模型：

```
face_detection_yunet_2023mar.onnx
```

运行：

```
python download_yunet_model.py
```

脚本会优先使用新版模型。

## 11. 第六个坑：活体脚本只显示 RGB，没有检测框 👀

相机通了，模型也不报错了，但画面上没有框。

这时问题通常是：

```
score_threshold=0.99
```

太严格。

原项目阈值是 `0.99`，真实相机画面里很容易检测不到人脸。

我们改成：

```
代码块
1   FACE_SCORE_THRESHOLD = 0.6
```

运行后画面左上角会显示：

```
代码块
1   Faces: 0
```

或者：

```
代码块
1   Faces: 1
```

如果一直是 `Faces: 0`：

- 人靠近一点
- 正对相机
- 增加光照
- 确认使用的是 `face_detection_yunet_2023mar.onnx`
- 可以把阈值继续降到 `0.3`

## 12. 三个脚本怎么跑

### 12.1 RGB-D 预览

```
python video_capture.py
```

会显示：

- RGB 图
- 深度灰度图
- 深度伪彩图

## 12.2 人体分割

```
python body_segmentation.py
```

会显示：

- Depth
- Segmentation
- Body Contour

可通过滑动条调节深度阈值：

```
dist1
dist2
```

默认阈值：

```
700mm - 900mm
```

## 12.3 人脸活体判断

```
python anti-spoofing.py
```

会显示：

- RGB 检测结果
- 深度伪彩图
- `Faces: N`
- `True` / `False` / `Undetermined`

终端会输出：

```
std: xx.xx
```

这个 `std` 是 5 个面部关键点的深度标准差。

# 13. 活体判断逻辑

核心代码思路：

```
```

```
std = np.std(dists)
```

其中 `dists` 是：

- 左眼
- 右眼
- 鼻尖
- 左嘴角
- 右嘴角

这 5 个点对应的深度值。

判断规则：

代码块

```
if std < 0:
    Undetermined
elif std < 5.0:
    False
else:
    True
```

含义：

- 深度无效：无法判断
- 深度变化很小：可能是照片/屏幕
- 深度变化明显：可能是真人

注意：这个阈值 `5.0mm` 不一定适合所有环境。你需要根据实际距离、光照、相机噪声重新调参。

## 14. 推荐调参流程

建议按这个顺序测：

1. 真人正脸，距离 0.5m
2. 真人正脸，距离 0.8m
3. 真人正脸，距离 1.2m
4. 手机屏幕照片，距离 0.5m
5. 打印照片，距离 0.5m

记录终端里的：

```
代码块
1   std: xx.xx
```

然后决定：

```
代码块
1   SPOOF_STD_THRESHOLD_MM = 5.0
```

是否要改成更合适的值。

## 15. 复现成功标志 ✅

如果你看到这些现象，说明基本复现成功：

### RGB-D 预览成功

```
代码块
1   Using software depth-to-color alignment.
2   Output RGB-D frame size: 640x480
```

窗口能显示 RGB 和深度图。

## 人体分割成功

能看到：

- 深度伪彩图
- 二值分割图
- RGB 上的人体轮廓

## 活体检测成功

画面左上角显示：

```
代码块
1   Faces: 1
```

人脸周围出现框，并显示：

```
代码块
1   True
```

或：

```
代码块alse
```

## 16. 最终经验总结 🌟

这次复现最大的经验是：

### 不要把问题都归因于算法

很多时候项目跑不起来，真正的问题在：

- 相机 SDK
- OpenCV 后端
- 模型版本
- 设备占用
- 图像尺寸
- I/O 性能
- 阈值设置

### 相机项目一定要先稳住数据流

建议复现顺序永远是：

1. 相机能打开
2. RGB 能显示
3. Depth 能显示
4. RGB 和 Depth 尺寸对齐
5. 再做人脸检测
6. 最后做活体判断

不要一上来就跑完整活体脚本。

那样一旦报错，你会不知道到底是相机、模型、深度还是显示窗口的问题。😄

## 17. 一句话版本

本次复现的关键路线是：

> 用 `pyorbbecsdk2` 替代 OpenCV `CAP_OBSENSOR`，统一 RGB-D 输出为 `640x480`，避免多进程抢相机，换用 YuNet 2023 模型，降低人脸检测阈值，再基于 5 个面部关键点深度标准差完成简单活体判断。

到这里，项目就从“能看 README 但跑不起来”，变成“能在 Jetson + Femto Mega 上一步步跑通”了。🎉